

操作系统原理 作业-2

第二章 Operating-System Structures（操作系统结构）

参考教材: *Operating System Concepts, 9th Edition (Silberschatz et al.)*

说明: 以下题目依据第二章核心知识点原创整理, 适合作为课后作业。

一、选择题（）

1. 以下哪一项不属于操作系统向用户和程序提供的基本服务？（ ）。
 - A. 程序执行（Program Execution）
 - B. I/O 操作（I/O Operations）
 - C. 软件模式设计（Software Schema Design）
 - D. 错误检测（Error Detection）
2. 从用户使用体验角度看，命令解释器（Shell）的主要作用是（ ）。
 - A. 负责完成进程调度
 - B. 作为用户与操作系统之间的接口，接收并解释命令
 - C. 直接管理磁盘中断
 - D. 替代内核执行所有系统调用
3. 系统调用（System Call）最合适的描述是（ ）。
 - A. 用户程序请求操作系统服务的编程接口
 - B. 内核之间相互通信的物理总线
 - C. 专门用于编译程序的命令集合
 - D. 用户程序内部的普通函数调用，与操作系统无关
4. 在系统调用分类中，创建和终止进程最直接对应于（ ）。

- A. 文件管理
- B. 设备管理
- C. 进程控制
- D. 信息维护

5. 关于单体内核（Monolithic Structure）的说法，正确的是（ ）。

- A. 所有内核功能都被严格限制在不同的用户进程中
- B. 各功能模块通常运行在同一内核地址空间中，性能较高但边界较弱
- C. 它无法支持系统调用
- D. 它比微内核一定更安全且更易扩展

6. 微内核（Microkernel）设计通常将下列哪类功能保留在内核中？（ ）

- A. 最小的进程/线程管理、地址空间管理和进程间通信
- B. 文件系统、网络协议栈和所有设备驱动
- C. 命令解释器和图形界面
- D. 所有系统服务与应用程序

7. 分层系统（Layered Approach）的主要思想是（ ）。

- A. 各层都可以任意调用任何其他层的函数
- B. 系统功能按层组织，上层建立在下层提供的服务之上
- C. 取消硬件抽象，直接面向物理设备编程
- D. 只适用于分布式操作系统

8. 可加载内核模块（Loadable Kernel Modules）的优势之一是（ ）。

- A. 必须在重新编译整个内核后才能扩展功能
- B. 兼顾内核内执行效率与一定的模块化扩展能力
- C. 能够完全消除所有内核错误
- D. 使所有服务都自动迁移到用户态

9. 虚拟机（Virtual Machine）的核心思想是（ ）。

- A. 让多个操作系统实例共享并抽象同一套底层硬件资源
- B. 让一个进程同时运行在多个 CPU 上
- C. 使用缓存代替主存
- D. 将系统调用全部转换成库调用

10. 关于系统启动（System Boot）的描述，最合理的是（ ）。

- A. 应用程序先加载，再由应用程序激活内核
- B. 引导程序负责初始化基本硬件环境并装入内核
- C. 启动过程只与 CPU 有关，与存储设备无关
- D. 一旦开机，操作系统无需任何装载过程即可自动存在于 RAM 中

二、简答题（）

1. 教材中将操作系统的服务分为“面向用户的便利服务”和“面向系统高效运行的服务”。请分别列举至少三项，并说明二者关注点的差异。

2. 什么是系统调用接口（API 与 system-call interface 的关系）？请说明应用程序、API、运行时库和内核之间的大致关系。

3. 请简述分层操作系统结构的优点与局限性。你认为在教学操作系统中采用分层设计有什么价值？

三、分析题（）

1. 某教学实验系统准备增加一个新的文件压缩服务。现有两种方案：

- (a) 把压缩功能直接编入内核；
- (b) 把压缩服务放到用户态，通过系统调用与内核交互。

请从安全性、性能、可维护性三个角度分析两种方案的差异，并说明在什么场景下你更倾向于选择哪一种方案。

2. eBPF 是一个非常强大的操作系统观测工具。本题目 eBPF 网络流量监控：基于 XDP 的包过滤与性能分析。请进行复现，并完成任务要求，截图展示结果，不用准备实验报告。

在麒麟高级服务器操作系统 V11（优先选择）上或者 Ubuntu22/24 操作系统上，随着网络流量的激增，如何高效、安全地监控和过滤网络包？传统防火墙或用户空间工具（如 iptables）可能引入延迟和开销，而 eBPF 提供了一种革命性的解决方案：允许用户在内核中运行自定义代码，实现零拷贝、高性能的包处理。这不仅提升了性能，还增强了安全性，特别是在麒麟 OS 或 Ubuntu 强调安全加固的环境中。

实验原理

- **eBPF 概述：**eBPF (extended Berkeley Packet Filter) 起源于 1992 年的 BPF，用于网络包过滤。2014 年扩展为 eBPF，支持更复杂的程序，如循环和函数调用。它在内核中运行字节码，经过验证器确保安全，避免崩溃。
- **XDP 机制：**XDP 是 eBPF 的一个钩子点，位于网络驱动接收包后、协议栈前。支持三种模式：PASS（继续处理）、DROP（丢弃）、TX（重定向）。在麒麟 OS 或 Ubuntu 上，XDP 可用于高性能网络应用，如 DDoS 防护。
- **eBPFmap (Maps)：**用于内核与用户空间共享数据，如计数器或配置。
- **相关麒麟 OS 或 Ubuntu 特性：**麒麟 V11 或 Ubuntu 22/24 集成了 eBPF 支持，通过 libbpf 库和 bcc 工具链。

背景知识

- **eBPF 历史：**从 BPF 到 eBPF 的发展，关键事件如 2015 年内核合并。
- **优势：**高性能（接近线速）、安全性（字节码验证）、灵活性（支持多种钩子）。
- **应用场景：**网络过滤、系统跟踪、容器安全。在麒麟 OS 或 Ubuntu 上，可用于智能运维和安全加固。

实验设计与实施

实验环境与工具

- **硬件环境：**麒麟高级服务器操作系统 V11 或 ubuntu 安装的服务器或虚拟机，至少 2 核 CPU、4GB RAM、千兆网卡。
- **软件环境：**
 - 内核版本：4.19+（麒麟 V11 或 Ubuntu 22/24 默认支持）。
 - 编译工具：clang 10+、llvm 10+、libbpf 0.4+。
 - 调试工具：bpftool、perf、tcpdump。
 - 包管理：yum install clang llvm libbpf-devel bpftool。
- **安全注意：**eBPF 程序在内核运行，确保代码无误；实验在非生产环境进行。
- **辅助工具：**文本编辑器（如 vim）、网络测试工具（如 ping、iperf）。

实验内容与步骤

1. **预备阶段（1 课时）：**
 - 复习 C 语言和网络基础。
 - 安装工具：`sudo yum update && sudo yum install clang llvm libbpf-devel bpftool`。
 - 测试环境：运行 `uname -r` 确认内核版本，`bpftool version` 检查工具。
2. **任务一：编写 eBPF 程序（XDP 过滤）**
 - 创建 `xdp_prog.c`，实现简单包过滤（丢弃非 IP 包）。
 - 添加 map 用于计数。
3. **任务二：编译和加载**
 - 使用 clang 编译：`clang -O2 -target bpf -c xdp_prog.c -o xdp_prog.o`。
 - 编写用户空间加载器 `loader.c`，使用 libbpf 加载程序。
 - 附加到接口：`ip link set dev eth0 xdp obj xdp_prog.o sec xdp`。
4. **任务三：测试与监控**
 - 生成流量：`ping` 或 `iperf`。
 - 监控计数：使用 bpftool 读取 map。
 - 分析性能：`perf stat` 测量延迟。
5. **任务四：扩展与优化**
 - 添加 TCP 端口过滤。
 - 比较 XDP 与传统 iptables 的性能。
6. **数据记录与分析**
 - 记录过滤前后流量、CPU 使用率。
 - 分析 eBPF 的开销。

核心代码 (C 语言和 eBPF)

```
// xdp_prog.c (eBPF 程序)
#include <linux/bpf.h>
#include <bpf/bpf_helpers.h>
#include <linux/if_ether.h>
#include <linux/ip.h>
#include <linux/tcp.h>
#include <linux/udp.h>

struct packet_stats {
    __u64 total_packets;
    __u64 ip_packets;
    __u64 tcp_packets;
    __u64 udp_packets;
    __u64 dropped_packets;
};

struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __uint(max_entries, 1);
    __type(key, __u32);
    __type(value, struct packet_stats);
} stats_map SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 1024);
    __type(key, __u32);
    __type(value, __u64);
} ip_count SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_PERCPU_ARRAY);
    __uint(max_entries, 1);
    __type(key, __u32);
    __type(value, __u64);
} per_cpu_packets SEC(".maps");

SEC("xdp")
int xdp_prog_simple(struct xdp_md *ctx) {
    void *data_end = (void *) (long) ctx->data_end;
```

```

void *data = (void *) (long) ctx->data;
struct ethhdr *eth = data;

__u32 key = 0;
struct packet_stats *stats = bpf_map_lookup_elem(&stats_map, &key);
if (stats) {
    __sync_fetch_and_add(&stats->total_packets, 1);
}

if ((void*)eth + sizeof(*eth) > data_end)
    return XDP_DROP;

// 过滤: 只允许 IP 包
if (eth->h_proto == __constant_htons(ETH_P_IP)) {
    struct iphdr *ip = (void*)eth + sizeof(*eth);
    if ((void*)ip + sizeof(*ip) > data_end)
        return XDP_DROP;

    if (stats) {
        __sync_fetch_and_add(&stats->ip_packets, 1);
    }

    // 更新 IP 计数
    __u64 *ip_cnt = bpf_map_lookup_elem(&ip_count, &ip->saddr);
    if (ip_cnt) {
        __sync_fetch_and_add(ip_cnt, 1);
    } else {
        __u64 init_val = 1;
        bpf_map_update_elem(&ip_count, &ip->saddr, &init_val, BPF_ANY);
    }

    // 检查 TCP/UDP
    if (ip->protocol == IPPROTO_TCP) {
        struct tcphdr *tcp = (void*)ip + (ip->ihl * 4);
        if ((void*)tcp + sizeof(*tcp) <= data_end) {
            if (stats) {
                __sync_fetch_and_add(&stats->tcp_packets, 1);
            }

            // 示例: 过滤特定端口
            if (tcp->dest == __constant_htons(80) || tcp->dest == __c

```

```

onstant_htons(443)) {
    return XDP_PASS;
}

} else if (ip->protocol == IPPROTO_UDP) {
    struct udphdr *udp = (void*)ip + (ip->ihl * 4);
    if ((void*)udp + sizeof(*udp) <= data_end) {
        if (stats) {
            __sync_fetch_and_add(&stats->udp_packets, 1);
        }
    }
}

// 示例: 丢弃特定 IP
if (ip->saddr == 0xC0A80101) { // 192.168.1.1
    if (stats) {
        __sync_fetch_and_add(&stats->dropped_packets, 1);
    }
    return XDP_DROP;
}

return XDP_PASS;
}

if (stats) {
    __sync_fetch_and_add(&stats->dropped_packets, 1);
}
return XDP_DROP;
}

```

```

char _license[] SEC("license") = "GPL";

```

```

// loader.c (用户空间加载器)

```

```

#include <bpf/libbpf.h>
#include <stdio.h>
#include <unistd.h>
#include <net/if.h>
#include <linux/if_link.h>
#include <signal.h>
#include <stdlib.h>
#include <string.h>

```

```

#include <arpa/inet.h>

volatile int running = 1;

void sig_handler(int sig) {
    running = 0;
}

int main(int argc, char **argv) {
    struct bpf_object *obj;
    struct bpf_program *prog;
    int prog_fd, ifindex;
    const char *ifname = "eth0";

    if (argc > 1)
        ifname = argv[1];

    ifindex = if_nametoindex(ifname);
    if (!ifindex) {
        perror("if_nametoindex");
        return 1;
    }

    obj = bpf_object__open_file("xdp_prog.o", NULL);
    if (libbpf_get_error(obj)) {
        fprintf(stderr, "Failed to open BPF object\n");
        return 1;
    }

    if (bpf_object__load(obj)) {
        fprintf(stderr, "Failed to load BPF object\n");
        return 1;
    }

    prog = bpf_object__find_program_by_name(obj, "xdp_prog_simple");
    prog_fd = bpf_program__fd(prog);

    if (bpf_xdp_attach(ifindex, prog_fd, XDP_FLAGS_SKB_MODE, NULL)) {
        fprintf(stderr, "Failed to attach XDP program\n");
        return 1;
    }
}

```

```

printf("XDP program attached to %s. Monitoring traffic...\n", ifname);

// 获取 mapFD
struct bpf_map *stats_map = bpf_object__find_map_by_name(obj, "stats_
map");
struct bpf_map *ip_count_map = bpf_object__find_map_by_name(obj, "ip_
count");
int stats_fd = bpf_map__fd(stats_map);
int ip_fd = bpf_map__fd(ip_count_map);

signal(SIGINT, sig_handler);

while (running) {
    sleep(5);

    // 读取统计
    __u32 key = 0;
    struct packet_stats stats;
    if (bpf_map_lookup_elem(stats_fd, &key, &stats) == 0) {
        printf("\n--- Packet Statistics ---\n");
        printf("Total packets: %llu\n", stats.total_packets);
        printf("IP packets: %llu\n", stats.ip_packets);
        printf("TCP packets: %llu\n", stats.tcp_packets);
        printf("UDP packets: %llu\n", stats.udp_packets);
        printf("Dropped packets: %llu\n", stats.dropped_packets);
    }

    // 读取前 10 个 IP 计数
    printf("\n--- Top IP Sources ---\n");
    __u32 ip_key;
    __u64 ip_value;
    void *prev_key = NULL;
    int count = 0;
    while (bpf_map_get_next_key(ip_fd, prev_key, &ip_key) == 0 && cou
nt < 10) {
        if (bpf_map_lookup_elem(ip_fd, &ip_key, &ip_value) == 0) {
            struct in_addr addr;
            addr.s_addr = ip_key;
            printf("%s: %llu packets\n", inet_ntoa(addr), ip_value);
        }
    }
}

```

```

        prev_key = &ip_key;
        count++;
    }
}

printf("\nDetaching XDP program...\n");
bpf_xdp_detach(ifindex, XDP_FLAGS_SKB_MODE, NULL);
bpf_object__close(obj);
return 0;
}

```

- **代码说明：**扩展 eBPF 程序为多 map 统计（总计、IP、TCP/UDP 计数、IP 源统计）。用户空间程序实时读取和显示统计数据。
- **代码说明：**eBPF 程序过滤并计数，用户空间程序加载、监控和分离。编译命令：`clang -O2 -target bpf -c xdp_prog.c -o xdp_prog.o`；运行：`./loader`。